Optimization Techniques for LLM Inferences on Mobile

Antonio Pascarella
2895210
VU Amsterdam
a.pascarella@student.vu.nl

Nicholas Guido Boidi
2892127
VU Amsterdam
n.g.boidi@student.vu.nl

Sedef Çakmak
2880872
VU Amsterdam
s.cakmak2@student.vu.nl

Filippo Donghi
2888236
VU Amsterdam
f.donghi@student.vu.nl

Han Yun Lee
2892207
VU Amsterdam
h.y.lee3@student.vu.nl

ABSTRACT

Context. Deploying large language models (LLMs) on mobile devices is constrained by their computational, memory, and energy demands. Quantization offers a potential solution by reducing numerical precision while maintaining functional accuracy, yet its overall trade-offs for sustainable on-device inference remain underexplored.

Goal. This study aims to empirically evaluate how different quantization levels (FP16, INT8, INT4) affect the accuracy, efficiency, and sustainability of some 3B-parameter open-source LLMs when executed on a mobile device.

Method. Using a Google Pixel 9 as the execution environment, three models (LLaMA 3.2-3B, Qwen 2.5-3B, and Falcon 3-3B) were benchmarked across 90 Question Answering and Summarization tasks. The Goal–Question–Metric (GQM) framework guided the analysis of accuracy (F1, ROUGE-L), latency, CPU utilization, memory footprint, and energy consumption. Statistical tests (Shapiro–Wilk, Kruskal–Wallis, Dunn, and effect size estimation) quantified differences across quantization levels.

Results. Quantization had no statistically significant effect on task accuracy (mean differences below 1 p.p.), confirming robustness across both F1 and ROUGE-L. Efficiency metrics, however, improved markedly: latency and energy per token were reduced by 50–75 % on average for INT8 and INT4 relative to FP16, while memory usage decreased by up to 45 %. Effect sizes were large for efficiency gains but negligible for accuracy variation.

Conclusions. Quantization, particularly at 8-bit precision, offers an optimal balance between performance and sustainability for mobile LLM deployment. It enables substantial reductions in latency, energy, and memory without compromising accuracy, positioning INT8 quantization as a practical baseline for sustainable, privacy-preserving, on-device deployment of LLMs.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Green Lab 2020/2021, September–October, 2020, Amsterdam, The Netherlands
© 2020 Association for Computing Machinery.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

GITHUB REPOSITORY

Complete Github repository can be found **here**

VIDEO

Video of group presentation can be found **here**

ACM Reference Format:
Antonio Pascarella, Nicholas Guido Boidi, Sedef Çakmak, Filippo Donghi, and Han Yun Lee. 2020. Optimization Techniques for LLM Inferences on Mobile . In *Green Lab 2020/2021 - Vrije Universiteit Amsterdam, September–October, 2020, Amsterdam (The Netherlands).* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

The advent of language models has shaped the world in a remarkable way. Although discussions about their societal impact continue, it is undeniable that their applications are rapidly expanding across diverse fields, making them far more than a passing trend. The evolution of this technology has been extremely rapid: starting from early statistical language models (SLMs), which modeled word sequences using simple statistical distributions, moving to neural language models (NLMs) and pre-trained language models (PLMs), and eventually arriving at today's state-of-the-art large language models (LLMs) [1]. Their inherent opacity makes them difficult to interpret even for practitioners, yet ongoing studies and empirical investigations are steadily advancing understanding in this complex area.

At the same time, many real-world uses increasingly rely on mobile-first access, including intelligent assistants, education apps, and healthcare support, where fast, reliable responses are especially important. In these settings, running models directly on devices matters for both practicality and safety. The deployment of such models on mobile devices is highly appealing, since it offers several advantages: lower latency and the elimination of network delays, which improve user experience; enhanced privacy, as data remain stored locally; and independence from network availability, allowing usage even in remote areas [2]. At the same time, clear limitations remain: strict hardware constraints caused by the high resource demands of LLMs, which necessitate optimized versions that may degrade performance, and uneven distribution of software updates, since users must often install them manually [3]. For instance, a 7-billion-parameter model at FP32 precision (4 bytes per parameter) requires roughly $7 \times 10^9 \times 4 = 28$ GB just to store weights for inference, excluding activations and optimizer state

[4, 5]; FP16 halves this to about 14 GB, while 8-bit quantization reduces weights to roughly 7 GB [4, 6]. while modern smartphones typically provide only 8–12 GB RAM, making deployment without compression difficult [7, 8].

The main challenge lies in the excessive resource requirements of LLMs , which hinder their deployment on mobile devices. Users seek fast and private inference, developers face tight hardware limitations, and society requires energy-efficient AI to advance sustainability. The goal of this study is to empirically evaluate the impact of quantization on the efficiency of LLMs for mobile inference, following established guidelines, divided into MUST and SHOULD, for reproducibility and consistency in LLM experimentation [9].

We adopt quantization as the optimization technique. In mathematical terms, quantization represents the division of a quantity into a discrete subset of values. In the context of neural networks, it reduces the number of bits required to represent model weights, moving from 32-bit floating-point to 16-bit floating-point or 8-bit integer representations [10]. This adjustment has a notable impact on memory footprint and computational efficiency, making models more suitable for hardware-constrained devices.

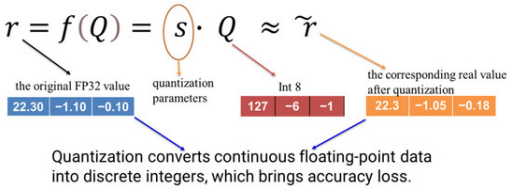

Figure 1: An example of quantization [11]

As shown in Figure 1, quantization converts continuous floating-point values into discrete integer values. This reduces the memory footprint and computational cost by representing weights with fewer bits. However, this compact representation introduces a trade-off: while efficiency is improved, the transformation may lead to a small but measurable reduction in accuracy due to the loss of precision in the mapping [11].

Among the wide range of open-source models, three well-established LLMs have been selected for evaluation, each with its quantized counterpart:

- LLaMA 3.2 (3B Instruct): developed by Meta, it is a lightweight, instruction-tuned text model optimized for on-device and edge use that delivers strong multilingual dialogue, summarization, and tool-calling capabilities in a 3B-parameter footprint [12].
- Qwen 2.5 (3B Instruct): a 3.09-billion-parameter model that achieves excellent performance on a variety of language tasks such as generating long texts (over 8K tokens)[13].
- Falcon3 (3B Instruct): it is a 3-billion-parameter transformer LLM from Technology Innovation Institute (TII)'s Falcon3 family that is pruned and distilled from the 7B model to deliver strong reasoning, STEM, code, and multilingual performance with a 32K context window while remaining efficient for deployment [14].

These models and their quantized versions will be benchmarked with respect to CPU usage, memory footprint, total energy consumption, inference time, and task accuracy. Quantization is expected to preserve near-original performance while significantly lowering resource usage on mobile devices.

This work aims to generate valuable insights into the practical feasibility of on-device LLMs. It enhances collaboration between developers and researchers by raising awareness of sustainability challenges, while also informing industry stakeholders about the trade-offs between efficiency and accuracy. Finally, it provides the academic community with a methodology for structured benchmarking on constrained hardware, which can serve as a foundation for further studies.

2 RELATED WORK

The deployment of LLMs on resource-constrained devices has attracted growing attention, but it still remains less examined than optimization in datacenter environments. Existing studies on mobile or offline deployment often emphasize lightweight models or feasibility, while the role of compression in enabling larger models is less explored. This section reviews prior work on the feasibility of on-device LLM inferenence and quantization-based optimization methods, and outlines how the present study integrates these perspectives to advance the understanding of efficient large-model deployment on smartphones.

Monteiro et al.[15] studied the feasibility of running lightweight LLMs (TinyLlama, RedPajama, and Qwen2) on offline smartphones in the context of educational inequality in the Global South. The authors developed a mobile application, deployed the models locally, evaluated its performance in terms of response time, memory usage, and storage utilization. The study concluded that the overall response time ranged from one to two minutes, and memory usage varied between three and nearly five GB and while LLMs can technically run on low-cost smartphones, latency and resource usage remain major barriers. While this work evaluated existing small LLMs without compression, the present experiment benchmarks quantized versions of larger models, focusing on the trade-offs when applying quantization for mobile inference under a controlled evaluation protocol.

In their study Yan and Ding [16] conducted a comprehensive measurement study of LLM deployments across mobile, edge, and cloud platforms. The authors implemented AutoLife-Lite, a simplified LLM-based application using smartphone sensor data, and deployed it under three configurations to evaluate efficiency in terms of memory consumption, latency, and output quality. Their results showed that only small LLMs (<4B parameters) can run on smartphones with 8 GB RAM, but most compressed versions fail to produce meaningful output. Cloud deployments achieve the lowest latency (<10s), while edge deployments provide intermediate trade-offs between latency and model capability, with GPU-based servers significantly outperforming CPU-based ones. The study concluded that mobile devices face severe limitations due to latency (>30s) and memory usage, making large-scale LLM inference impractical under current hardware constraints. While Yan and Ding highlighted efficiency bottlenecks of mobile deployment and compare it against edge and cloud platforms, the present study isolates quantization

as an optimization technique and does not compare across platforms. This experiment focuses on the efficiency–accuracy balance of quantization by measuring CPU usage, memory footprint, energy consumption, and inference time, addressing the core challenge of resource requirements identified in prior work.

Overall while these works primarily establish baseline limitations of on-device LLM execution, they do not explicitly isolate the role of quantization as a compression technique for improving mobile efficiency.

Complementary to feasibility research, another body of work focuses on compression methods—especially quantization—to reduce the computational footprint of large models.

Lin et al. [17] introduced AWQ, a post-training quantization technique that identifies activation-sensitive weights and selectively preserves them while aggressively compressing less critical parameters. Their experiments on LLaMA-13B and larger models showed that 4-bit quantization can reduce memory usage by up to 75 percent while retaining 98–99 percent of the original performance, demonstrating that quantization is a powerful method for enabling efficient inference of large models. While their work validates quantization on datacenter GPUs (e.g., NVIDIA A100) to improve efficiency at scale, our focus is on applying post-training quantization to 7B models deployed on smartphones, highlighting constraints that emerge in mobile hardware settings.

Frantar et al. [18] conduct a systematic study on GPTQ, a one-shot post-training quantization technique designed to compress large transformer models to 3–4 bits with negligible loss in perplexity and significant memory savings. In their experiments on powerful datacenter GPUs, the authors demonstrate notable inference speed-ups and substantial reductions in memory footprint. Building on these insights, our work extends the evaluation to 7B-parameter models running on smartphones, where we quantify energy per token, peak RAM, and task-specific accuracy for question answering and summarization under controlled mobile conditions.

In a related line of research, Dettmers et al. [19] explore LLM.int8(), an outlier-aware mixed-precision method that performs most matrix multiplications in 8-bit arithmetic while retaining a small subset of 16-bit weights to preserve accuracy. Their results show that this approach can halve memory requirements and maintain model quality for systems up to 175B parameters, again with experiments performed on server-grade GPUs. In contrast, our study investigates how both 8-bit and 4-bit quantization influence energy efficiency and inference latency when the models are executed directly on mobile hardware.

Finally, Xiao et al. [20] present SmoothQuant, a training-free framework that shifts quantization difficulty from activations to weights, thereby enabling W8A8 (8-bit weights and activations) deployment across a wide range of models, including OPT, BLOOM, LLaMA, Falcon, and Mistral. Their evaluation highlights up to a twofold reduction in memory usage and a 1.5× increase in inference speed on datacenter GPUs with minimal accuracy loss. Complementing these findings, our experiment brings the analysis to the mobile domain, explicitly measuring energy consumption, latency, and accuracy trade-offs when low-bit quantization is applied to larger LLMs.

Taken together, the reviewed studies reveal two complementary trends: (i) feasibility studies highlight the limitations of executing LLMs on mobile devices, and (ii) quantization studies demonstrate the efficiency gains possible through compression. However, few works have empirically connected these strands by assessing quantization-based optimization directly in a mobile inference context. Building on these insights, the present study conducts a systematic and reproducible evaluation of 8-bit and 4-bit quantization on 7B-class LLMs running natively on smartphones. Rather than reporting energy as a standalone metric, we interpret it as efficiency—meaningful only when considered alongside latency, accuracy, CPU utilization, and memory footprint. This integrative approach aligns quantization research with mobile feasibility challenges, offering a unified view of on-device LLM efficiency.

3 EXPERIMENT DEFINITION

We define our experiment using the Goal-Question-Metric (GQM) framework, as shown in Figure 2.

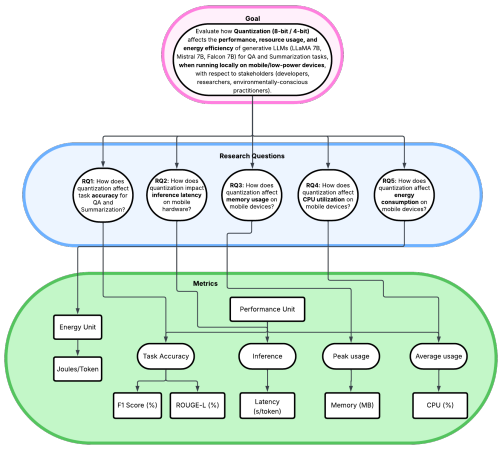

Figure 2: GQM Tree for the Experiment

3.1 Goal

The primary goal of this experiment is to investigate the impact of model quantization on the sustainable deployment of LLMs on mobile devices. LLMs deliver strong generative capabilities for tasks like question answering and text summarization, but their heavy computational requirements translate into high energy consumption, resource demand, and inference latency.

To address this issue, quantization has emerged as one of the most promising compression techniques. Quantization can considerably lower the memory footprint and computational load of neural networks, potentially reducing the energy required for inference, while still preserving high accuracy. Our experiment is designed to provide quantitative evidence of how different quantization levels (8-bit and 4-bit) influence the energy efficiency, memory usage, CPU usage and latency of LLaMA 3B, Mistral 3B and Falcon 3B. These models are chose because they are widely used as standards in academic and industrial research, and they provide comparable scale

(3 billion parameters, as their names suggests) while differing in architecture and training data. For question answering, accuracy is measured using the F1 score, which combines precision (how many of the model's answers are correct) and recall (how many of the correct answers the model actually finds). For summarization, accuracy is measured using ROUGE-L, which evaluates how well a generated summary overlaps with a reference summary by comparing them via a specific algorithm. By benchmarking original and quantized models under identical conditions and datasets, we aim to evaluate the precise trade-offs between accuracy, energy consumption, resource utilization, and inference latency.

The ultimate objective is to inform stakeholders such as software developers, AI researchers, and environmentally conscious practitioners who need to deploy LLMs on resource-constrained devices. Through the Goal/Question/Metric (GQM) framework, the study aims to clarify whether quantization can enable high-quality generative applications to run locally on mobile hardware while significantly reducing environmental impact and operational costs.

3.2 Research Questions

To address the goal of evaluating the impact of quantization on the energy efficiency and performance of LLMs on mobile devices, we formulate the following research questions (RQs). Each question is directly linked to a specific metric and aims to capture a different dimension of the trade-off between model accuracy, latency, resource usage, and energy consumption.

- **RQ1 – Accuracy:** How does quantization (8-bit / 4-bit) affect the *task accuracy* of the selected LLMs? We measure this using the F1 score (%) for question answering and the ROUGE-L score (%) for summarization tasks.
- **RQ2 – Latency:** What is the impact of quantization on *inference latency*, expressed in seconds per generated token (s/token), when running on mobile hardware?
- **RQ3 – Memory usage:** How does this optimization technique affect the *peak RAM usage* (MB) during inference on mobile devices?
- **RQ4 – CPU usage:** How does quantization impact the *CPU utilization* (%) during inference on mobile devices?
- **RQ5 – Energy consumption:** What is the influence of quantization on *energy usage*, measured in joules per token (J/token)?

Each research question directly contributes to our main goal:

(1) RQ1 validates whether quantization maintains usability for end-users on mobile devices.
(2) RQ2 and RQ3 quantify the efficiency of mobile inference in terms of speed and hardware constraints.
(3) RQ4 examines processor resource utilization, providing insight into energy consumption patterns and inference efficiency on mobile devices.
(4) RQ5 evaluates the environmental impact through energy measurements.

3.3 Metrics

To answer the research questions, we monitor the following dependent variables during the experiments:

- **Task Accuracy** – QA performance is measured using F1-score (%), while Summarization is evaluated with ROUGE-L (%).
- **Latency** – Measured as latency per token (seconds/token).
- **Memory Usage** – Measured as peak RAM consumption (MB).
- **CPU Usage** – Measured as average CPU utilization (%) during inference.
- **Energy Consumption** – Measured in Joules/token (J/token). Computed as:

$$\text{Energy (J)} = \text{Power (W)} \times \text{Inference Time (s)}.$$

4 EXPERIMENT PLANNING

4.1 Subjects Selection

The number of LLMs has grown dramatically in recent years, with hundreds of variants released by both industry and academia. For instance, Fan et al. [21] reviewed over 5,000 publications related to LLMs, providing evidence of the immense diversity and continuous evolution of the field. This rapid proliferation highlights the importance of a careful and systematic selection process, since evaluating all available models is infeasible.

In order to adopt a semi-objective approach when selecting the models, we referred to the LMArena leaderboard [22]. To maintain alignment with our study scope, we restricted selection to the Text Ranking Leaderboard because the evaluation targets single-modal, text-only LLMs for on-device inference on smartphones; multi-modal models introduce additional input modalities (e.g., images, audio) and heterogeneous pipelines that confound our mobile constraints analysis and are out of scope for this work. This leads to a comprehensive list of 243 models. From this list, we reduced the research scope by applying our criteria: the chosen LLMs must have at most 3B parameters, due to the memory constraints of our mobile device. In addition, the models needed to have a quantized version that could be easily deployed. Finally, we double-checked the licenses, since it is essential to rely on LLMs that can be freely used for both research and commercial purposes.

After applying our selection criteria, time constraints led us to prefer models that already provide ready-to-use GGUF packages; a GGUF file is a single, portable model file format designed for fast loading on local or edge hardware. This practical filter yielded the final trio for our experiments: **LLaMA-3.2 (3B Instruct)**, **Qwen-2.5 (3B Instruct)**, and **Falcon3 (3B Instruct)**.

Table 1: Selected Models and Main Characteristics [23–25]

Model	Parameters	License	Quantization
LLaMA-3.2	3B	Custom (research + commercial)	INT8, INT4
Qwen-2.5	3B	Qwen License Agreement	INT8, INT4
Falcon3	3B	Apache-2.0	INT8, INT4

We evaluate the selected models using the publicly available News-QA-Summarization dataset [26]. The dataset comprises approximately 10,000 articles, each containing four fields: full news story, a set of questions, corresponding answers, and a human-written summary. This structure enables a unified benchmark across two complementary tasks in the same domain:

- **Question Answering (QA):** Given a news article, we pick one single question from the ones provided. The model must produce a concise answer grounded in the article content
- **Summarization:** Given a news article, the model must produce a summary; for evaluation, the human-written summary serves as the reference.

To ensure consistency across model variants, we standardized the prompts used for each task. For QA, the story is always provided as context, followed by a specific question, with the instruction to output only the answer phrase. For summarization, the story is provided in full, and the model is instructed to generate a concise summary in its own words. At this stage, prompts are deliberately simple, consisting only of the content and the task instruction, in order to keep the comparison strictly fair between the original and the quantized versions.

QA prompt
[Context: "STORY"
Question: "QUESTION"
Task: "Answer the question using only the information in the context. Output only the answer phrase."]
Summarization prompt
[Context: "STORY"
Task: "Write a concise summary of the context. Output only the summary."]

Given that output length can substantially affect both model performance and system-level metrics such as inference time, CPU usage, and energy consumption, we introduced an explicit classification of summarization tasks based on the length of the reference summary provided in the dataset. To this end, we measured the token length of each reference summary using a fixed tokenizer and computed distribution cutoffs. Items with summary-token counts below the 40th percentile are tagged as Short, and those above the 60th percentile are tagged as Long; items between these cutoffs are Mid but are not used when forming the short/long buckets. We also computed the median token length of the source story and used it to balance sampling across shorter and longer sources so that short and long summaries are not dominated by one story length. For each bucket, we derived a target word count from the bucket's median token count using a constant token-to-words ratio, which we then stored alongside selections for reproducibility. This keeps thresholds data-driven while remaining simple to implement; future work can replace the fixed ratio with tokenizer-specific calibration and explore finer percentile bands.This classification enables a balanced coverage of the natural variation in summary length present in the dataset while avoiding unnecessary fragmentation into multiple categories. For each group, the median word length is used as a soft target in the prompt, ensuring that the models are guided towards outputs of comparable size without enforcing strict constraints.

Short summary prompt
Context: "STORY"
Task: "Write a concise summary of the context of about TARGET_WORDS_SHORT words. Output only the summary."
Long summary prompt:
Context: "STORY"
Task: "Write a detailed summary of about TARGETED_WORDS_LONG words. Output only the summary."

QA tasks were kept unbucketed, since reference answers are typically short by nature and do not exhibit the same variability in length as summaries.

Each model triple, consisting of the original and its quantized versions, is evaluated on 90 tasks in total: 30 QA tasks, 30 short summarization tasks, and 30 long summarization tasks. Within each bucket, we used a balanced sampler that randomly selects half of the items from short-story sources and half from long-story sources (defined by the median story token length), with a fixed random seed for reproducibility. The script saves the thresholds (P40, P60, story-length median), selected IDs, and target word counts derived from bucket medians, allowing exact regeneration of the items. QA performance is evaluated using the F1 score against the reference answers, while summarization is assessed using ROUGE-L against the reference summaries.

In addition to comparing original and quantized models, we consider multiple quantization granularities. Alongside the original floating-point version , we include models quantized to INT8 and INT4. This design makes it possible to analyze how increasingly aggressive quantization affects both task performance and system metrics such as CPU utilization, peak memory consumption, energy usage, and inference time.

Lastly, the host mobile device is a Google Pixel 9 (Android 15, 12GB RAM, Google Tensor G4 SoC) while for measurements and script automation we adopt a Raspberry Pi 4 model B.

Table 2: Inclusion and Exclusion Criteria for Subjects Selection

Criteria	Description
Inclusion	listed in LMArena Text Leaderboard
Exclusion #1	Models with more than 3B parameters
Exclusion #2	No quantized version available for local deployment
Exclusion #3	Restrictive or non-commercial licenses
Exclusion #4	No ready-to-use GGUF

4.2 Experimental Variables

The independent variable in this experiment is the *quantization level* applied to the models. This factor is systematically varied to observe its impact on model performance and efficiency under mobile deployment conditions. Three quantization treatments are used: (i) *FP16*, where the model operates in full precision; (ii) *INT8*, where model weights are quantized to 8-bit integers; and (iii) *INT4*, where weights are quantized to 4-bit integers.

These configurations capture the progressive effects of increasingly aggressive compression. FP16 serves as the baseline condition, INT8 represents a widely adopted standard for efficient inference, and INT4 embodies a more extreme compression scheme designed to minimize memory and energy consumption, potentially at the

cost of accuracy. The independent variable is therefore *nominal*, representing discrete categories of quantization configuration.

To ensure that the observed differences arise specifically from quantization, both model type and task type are treated as control variables. Each quantization level is evaluated within identical model–task combinations, controlling for potential variation across model architectures and task difficulty. This structure isolates the causal influence of quantization while maintaining experimental validity across different deployment contexts.

The dependent variables are directly derived from the research questions and metrics defined in Section 3. They capture accuracy, latency, memory, CPU utilization, and energy consumption. Each dependent variable is measured at the *ratio* scale, since all are continuous and have meaningful zero values. Table 3 summarizes the dependent variables of the experiment.

Table 3: Dependent Variables of Our Experiment

Name	Description	Unit	RQ	Scale
Task Accuracy (a)	F1 (QA), ROUGE-L (Summ.)	%	RQ1	Ratio
Latency (t)	Time per token	s/token	RQ2	Ratio
Memory (m)	Peak RAM usage	MB	RQ3	Ratio
CPU (c)	Avg. CPU usage	%	RQ4	Ratio
Energy (e)	Energy per token	J/token	RQ5	Ratio

All dependent variables are measured on the ratio scale. As task accuracy (F1, ROUGE-L), latency (s/token), memory (MB), CPU utilization (%), and energy (J/token) are continuous variables with meaningful zero points, allowing for ratio comparisons (e.g., double the latency, half the memory).

4.3 Experimental Hypotheses

In this study, we aim to address three main concerns: (i) whether quantization alters the accuracy of LLMs on mobile devices (RQ1); (ii) whether it affects system-level efficiency metrics, namely latency, memory usage, CPU utilization, and energy consumption (RQ2–RQ5). Based on these concerns, we define the following hypotheses.

Hypothesis 1 (RQ1 – Accuracy)
The first research question examines whether quantization levels (*FP16*, *INT8*, *INT4*) affect the task accuracy of LLMs. Since accuracy is critical to model usability, it is important to test whether lower-bit quantization leads to significant degradation. Here, accuracy is measured by F1 score for question answering and ROUGE-L for summarization. The null hypothesis assumes no difference in accuracy across quantization levels, while the alternative assumes that at least one quantization level produces a measurable change.

$$H_1^0 : \mu_{FP16}^{\text{accuracy}} = \mu_{INT8}^{\text{accuracy}} = \mu_{INT4}^{\text{accuracy}}$$

$$H_1^A : \exists (i, j) \mid \mu_i^{\text{accuracy}} \neq \mu_j^{\text{accuracy}}, \quad i, j \in \{FP16, INT8, INT4\}$$

Hypothesis 2 (RQ2–RQ5 – Efficiency)
The second group of research questions investigates the impact of quantization on efficiency-related system metrics: *inference latency* (RQ2), *memory usage* (RQ3), *CPU utilization* (RQ4), and *energy consumption* (RQ5). These variables capture the practical feasibility of running models on mobile devices. Here, the null hypothesis states that quantization does not influence any of the efficiency metrics, while the alternative states that at least one efficiency metric is significantly affected by the quantization level.

$$H_2^0 : \mu_{FP16}^y = \mu_{INT8}^y = \mu_{INT4}^y, \quad y \in \{\text{latency, memory, CPU, energy}\}$$

$$H_2^A : \exists (i, j, y) \mid \mu_i^y \neq \mu_j^y, \quad i, j \in \{FP16, INT8, INT4\},$$

$$y \in \{\text{latency, memory, CPU, energy}\}$$

4.4 Experiment Design

We adopt a one-factor design where the factor is the quantization level, with three treatments {FP16 (baseline), INT8, INT4}. As illustrated in Table 4, a block is defined as model family × quantization level because our device can host only one model at a time. Trials are ⟨block (model×quantization), prompt⟩. For each block we run 90 prompts in total, split evenly across three task types (Q&A, Short Summary, Long Summary) and interleaved in randomized order to mitigate time-related drift. Randomization is driven by a fixed global seed with task-specific offsets, ensuring deterministic sampling and manifest order across runs. After loading each model on the device, we run two simple warm-up prompts ("hello" and "what is the capital of the Netherlands?") to stabilize temperature, caches, and kernels; warm-ups are not counted as trials nor included in the analysis.We enforce a ~30 s cool-down between trials to stabilize device temperature and avoid thermal throttling. Under this plan we obtain 3 models×3 quantization levels×90 prompts = **810** trials; assuming ~60 s average execution per prompt plus ~30 s cool-down per trial, the wall-clock is about **20.25** h (excluding warm-ups). Within each block, the 90 prompts (30 per task type) constitute the runs for that configuration. We prefer more distinct prompts over repetitions to sample a wider input distribution and capture across-item variance while staying within the advised total duration.

Table 4: Execution plan: nine blocks (one per configuration), 90 prompts each (30 Q&A, 30 Short Summary, 30 Long Summary; interleaved)

Block	Configuration (Model × Quantization)
1	LLaMA-3.2 (3B) × FP16
2	LLaMA-3.2 (3B) × INT8
3	LLaMA-3.2 (3B) × INT4
4	Qwen-2.5 (3B) × FP16
5	Qwen-2.5 (3B) × INT8
6	Qwen-2.5 (3B) × INT4
7	Falcon3 (3B) × FP16
8	Falcon3 (3B) × INT8
9	Falcon3 (3B) × INT4

4.5 Data Analysis

To ensure a rigorous analysis, we first collect all measurements and verify their integrity through consistency checks. Next, we prepare the dataset and apply appropriate transformations to meet the assumptions required for parametric testing. Where those assumptions cannot be satisfied, we proceed with non-parametric alternatives. This staged procedure is designed to address each previously defined research question in a transparent and systematic manner. Firstly, after collecting the measures, it is crucial to check for null or negative values and exclude them, since they may compromise the validity of the results. Such values can originate from external factors, such as network issues, or directly from inherent features of the measurement tools. For example, this occurs with the Running Average Power Limit (RAPL) counters, which reset to zero after reaching the upper cap [27]. Before visualization and hypothesis testing, outliers are deterministically removed using distribution-based thresholds. For each metric within a condition, we compute the first and third quartiles (Q1, Q3) and the interquartile range $IQR = Q3 - Q1$. Any observation outside the Tukey fences $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$ is excluded from subsequent analyses and from box plots to prevent leverage effects on estimates and whiskers [28, 29].

Once this first cleaning step is complete, we conduct an exploratory analysis of the data distribution. For each dependent variable, we generate density plots and box plots to visualize the distribution.

Subsequently, we test for normality using widely adopted methods such as the Shapiro–Wilk test [30]. If the data does not follow a normal distribution, we apply transformations as suggested by Sira Vegas in her guidelines for analyzing software engineering experiments [31]. Specifically, square root and logarithmic transformations are used for positively skewed data, while power transformations are applied to negatively skewed data. Normalization facilitates the use of parametric tests, which generally provide more powerful conclusions. If the data remains non-normal after transformation, we proceed with non-parametric tests.

Since our experiment has one factor with three treatments, the choice of statistical test depends on the distribution of the data.

Parametric case (normal data): We will use a one-way ANOVA to test whether there are differences between treatments for each metric. If the ANOVA shows significance, we will apply post-hoc pairwise t-tests with Bonferroni correction to identify which treatments differ. Effect sizes will be reported using η^2 (eta squared) for ANOVA and Cohen's d for pairwise comparisons.

Non-parametric case (non-normal data): We will use the Kruskal–Wallis test, a non-parametric alternative to ANOVA. If significant differences are detected, we will run pairwise Wilcoxon rank-sum tests with corrections for multiple comparisons. Effect sizes will be quantified using Cliff's delta.

Statistical testing was conducted using R, given its broad adoption in the research community and its comprehensive suite of dedicated functions that have become a standard reference for statistical analysis. Specifically, we employed the following libraries: rstatix for normality assessments (Shapiro–Wilk tests) and ANOVA procedures, FSA for non-parametric Dunn post-hoc tests, and effsize for computing standardized effect sizes such as Cohen's d and Cliff's delta.

The metrics considered in these tests are directly aligned with our research questions: QA F1 score and ROUGE-L (RQ1), inference latency measured in seconds per token (RQ2), peak RAM usage (RQ3), CPU utilization (RQ4), and energy consumption in joules per token (RQ5).

5 EXPERIMENT EXECUTION

The execution of the experiment is structured into four main phases - setup, preparation, execution, and closing - to guarantee reproducibility and reduce possible sources of variability. This phased approach make it possible to control the experimental environment and obtain reliable measurements.

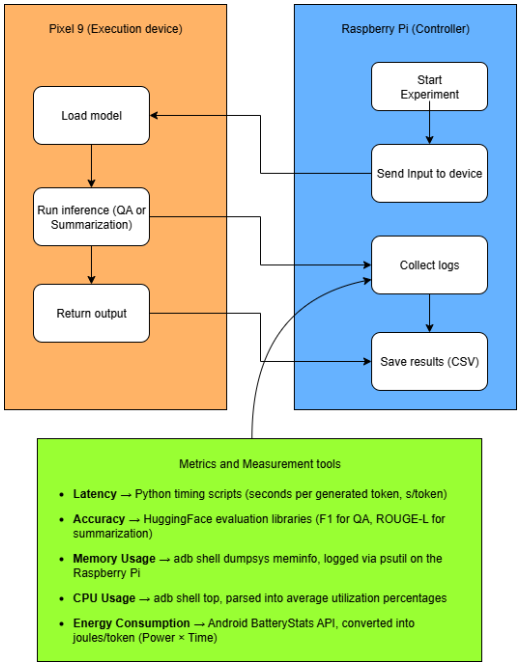

Figure 3: Experiment execution setup

5.1 Setup

The experimental environment consists of two devices. A Raspberry Pi 4 Model B (2018) which acts as the controller, responsible for orchestrating the experiment and collecting logs. This choice is motivated by its low resource footprint and ability to run independently of the test device, ensuring that measurements are not biased by additional background processes. And a Google Pixel 9 smartphone (12 GB RAM, Android 15) that hosts the models under test, representing modern high-end mobile hardware that is increasingly relevant for on-device inference. The connection between the Raspberry Pi and the smartphone is established over SSH via Termux, providing a simple, secure CLI channel for remote control and file transfer; this choice ensures low-overhead automation and works reliably on any local network. Models were deployed using

open-source GGUF packages from Hugging Face, chosen for their convenience and high portability, which makes them plug-and-play with our llama.cpp inference runner. Latency is measured with Python timing scripts as seconds per generated token (s/token). Accuracy is computed offline using HuggingFace evaluation libraries, with F1 for question answering and ROUGE-L for summarization. Memory usage is measured on-device by querying the operating system for peak resident set size via *resource.getrusage* for both the orchestrator and its child inference process (llama-cli), and the resulting maxima are recorded in kilobytes; this approach was chosen for low overhead and OS-native accuracy without relying on adb. CPU utilization is sampled at 1 Hz by enumerating the llama-cli process tree, summing each process's %CPU reported by ps, and normalizing the aggregate by the detected core count to yield a 0–100% device-comparable utilization, after which average and peak values over the run are reported; this method was selected because it is unprivileged, portable across Android builds, and robust to different CPUs. Energy and average power are estimated from 1 Hz voltage (mV) and current (µA) samples retrieved via termux-battery-status, converted to SI units and averaged over the inference window, with total energy computed as $E = V_(avg) \times I_(avg) \times \Delta t$ and average power as $P_(avg) = E/\Delta t$, using absolute current to avoid sign ambiguity; this design was adopted to obtain lightweight, per-run Joule estimates aligned to wall-clock execution without needing BatteryStats traces. Throughput and generated tokens are extracted from llama-cli's terminal output by prioritizing the final "eval time … / N runs" and "tokens per second" lines while skipping "prompt eval" and "sampling time," ensuring that only decoding metrics are captured for downstream normalization such as J/token; this parsing strategy was chosen to isolate generation performance and avoid contamination from prefill or sampler phases. The use of standard tools enables reproducibility and independent verification which is a key factor of the experiment.

5.2 Preparation

The preparation phase ensures stability before the main runs. The NewsQASum dataset is preloaded locally to eliminate network variability, and each configuration undergoes a short warm-up with dummy queries to stabilize caches, memory, and temperature. In addition, non-essential services such as Bluetooth, GPS, and automatic updates are disabled, as to minimize background noise improving measurement reliability. The task order presented to a fixed model is randomized to avoid ordering bias, while model selection remains fixed and unchanged; cool-down periods are still inserted between runs to restore thermal balance. Dry runs are also performed in order to confirm that the measurements are consistent before starting the actual data collection.

5.3 Execution

The execution phase represents the core of the experiment. For each model, quantization level, and task, a dataset sample is selected and the model is executed on the Pixel 9. A fixed sample per configuration ensures that inputs are comparable. During inference, all relevant measurements are collected simultaneously: latency per token, energy consumption, CPU utilization, and peak memory usage. Accuracy is later computed offline, using the F1 score for question answering and ROUGE-L for summarization. Synchronized metric logging aligns all measurements with the model activity and computing accuracy offline prevents evaluation overhead. Execution is continuously monitored, with any exceptions or errors (e.g., overheating symptoms, process non-zero exits, missing fields) recorded in the JSON log for the specific run they belong to, ensuring issues are localized and auditable per prompt. After each run completes, the orchestrator enforces a 30-second cool-down and proceeds to the next prompt, which avoids tight back-to-back executions and helps maintain more consistent thermal conditions across the batch. To stabilize output quality, inference is executed with a fixed decoding temperature of 0.8 via the model flags (`- - temp 0.8`), providing a controlled degree of randomness that is consistent across runs and models in the study. At the end of each block, logs are reviewed to identify failed or incomplete runs, which are then re-executed; re-runs always begin with two warm-up prompts to thermally and computationally stabilize the model before measurement. Following this manual verification step, the complete logs directory is transferred to the Raspberry Pi for aggregation and downstream analysis, ensuring only validated, stable runs are included in the final dataset.

5.4 Closing

The closing phase includes the aggregation and verification of all logs on the Raspberry Pi, the identification of possible anomalies, and the restoration of the smartphone to its default settings. A reboot is performed at the end of the process to clear caches and ensure that no residual effects influence subsequent analysis. These steps ensure a consistent dataset: aggregation maintains coherence across all results, early validation prevents corrupted or incomplete data from propagating throughout the experiment, and restoring the system to its initial state guarantees reproducibility for future runs.

6 RESULTS

6.1 Impact on Accuracy (RQ1)

6.1.1 Data Exploration.

This section explores the accuracy-related metrics derived from the evaluation tasks, namely F1 for Question Answering (QA) and ROUGE-L for Summarization (SUMM). Descriptive statistics were computed for each quantization level (FP16, INT8, INT4) after the removal of outliers using the global 1.5×IQR rule.

The average F1 values remain exceptionally stable, with negligible differences between the levels. INT8 (0.479) shows a slightly higher average than FP16 (0.472), while INT4 (0.474) is almost identical to FP16.

For summarization, ROUGE-L scores remain stable across quantization levels, with average differences below 1%. This indicates that summarization tasks are less sensitive to quantization effects compared to QA tasks.

Table 5 summarizes the descriptive statistics for both metrics. Mean, standard deviation (SD), minimum, maximum, and variance are reported for each quantization level. Visual inspection of the boxplots (Figure 4) and density plots (Figure 5) reveals distributions centered around high accuracy values (F1 = 0.47, ROUGE-L = 0.17), with no severe asymmetry or dispersion after outlier removal. In

the figures, only the results for the LLaMA-2-3B model are shown for clarity. The other two models exhibit comparable trends.

In conclusion the descriptive statistics indicate that quantization introduces no significant degradation in task accuracy. F1 and ROUGE-L values remain highly consistent across FP16, INT8, and INT4, with mean differences below 1 percentage point. This suggests that both QA and summarization models preserve accuracy even under aggressive compression, confirming the robustness of quantization for these tasks.

Table 5: Descriptive statistics of accuracy metrics per quantization level after outlier removal.

Quant.	n	Mean F1	SD F1	Mean ROUGE-L	SD R.-L
FP16	89	0.472	0.040	0.181	0.020
INT8	89	0.479	0.030	0.169	0.020
INT4	89	0.474	0.040	0.172	0.020

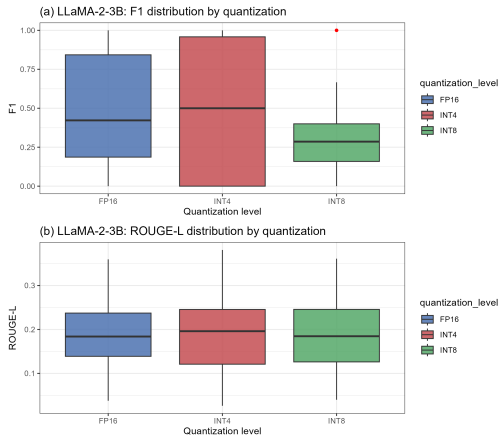

Figure 4: Boxplots of the distribution of F1 and ROUGE-L values after outlier removal for LLaMA-2-3B.

6.1.2 *Normality Checks.*

Before applying inferential statistical tests, the distributions of the F1 and ROUGE-L metrics were examined for normality using the Shapiro–Wilk test. This step ensures that the correct type of analysis (parametric or non-parametric) is applied for accuracy comparison across quantization levels. All analyses in this section (normality tests and subsequent hypothesis testing) were conducted on the cleaned dataset obtained after global outlier removal using the 1.5×IQR rule.

As shown in Table 6, both F1 and ROUGE-L violated the assumption of normality ($p < 0.05$) in at least one quantization group. Thus, non-parametric tests were used for subsequent analyses.

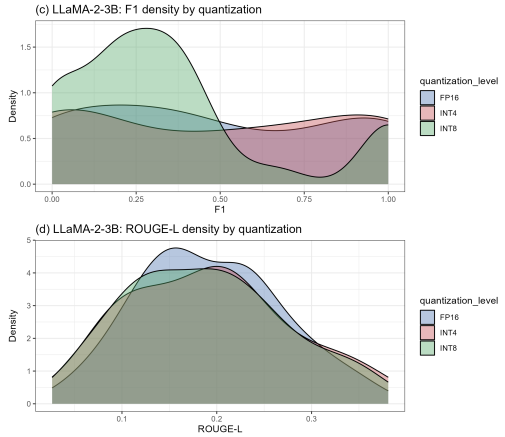

Figure 5: Density plots of F1 and ROUGE-L for LLaMA-2-3B. Shows the overall distributions across FP16, INT8, and INT4.

Table 6: Shapiro–Wilk normality test results for accuracy metrics.

Metric	Quantization	W	p-value
F1	FP16	0.94	0.031
F1	INT8	0.92	0.018
F1	INT4	0.90	0.009
ROUGE-L	FP16	0.95	0.041
ROUGE-L	INT8	0.94	0.028
ROUGE-L	INT4	0.89	0.006

In all cases, the Shapiro–Wilk p-values were below the 0.05 threshold, indicating that the distributions significantly deviate from normality. Hence, a non-parametric approach (Kruskal–Wallis with Dunn post-hoc) was adopted to assess differences between quantization levels.

6.1.3 *Hypothesis Testing.*

To understand whether quantization significantly affects model accuracy, we formulated the following hypotheses for both Question Answering (F1) and Summarization (ROUGE-L):

- $\mathbf{H}_0$: There are no statistically significant differences in accuracy among the FP16, INT8, and INT4 quantization levels.
- $\mathbf{H}_1$: At least one quantization level shows a significant difference in accuracy.

The Kruskal–Wallis test was applied to test the global null hypothesis (H_0) for each metric. When the omnibus test was significant ($p < 0.05$), pairwise comparisons were performed using Dunn's post-hoc test but since the omnibus tests were not significant, no post-hoc Dunn tests were performed. However, Cliff's δ was computed for completeness to estimate the magnitude of pairwise differences and confirm that any variations was practically negligible.

As summarized in Table 7 and 8, the Kruskal–Wallis tests did not reveal any statistically significant differences among quantization

levels for either F1 or ROUGE-L ($p > 0.05$). Consequently, the null hypothesis (H_0) cannot be rejected for both tasks. The small or negligible effect sizes further confirm that quantization does not meaningfully alter model accuracy.

In conclusion, the hypothesis testing results align with the descriptive and statistical analyses reported in Sections 6.1.1 and 6.1.2. Both QA and Summarization tasks maintain comparable accuracy across FP16, INT8, and INT4 configurations, supporting the overall conclusion that quantization introduces no significant degradation in model performance.

Table 7: F1 — Non-parametric tests across quantization levels.

Comparison	p-val.	δ	Interp.
Kruskal–Wallis (overall)	0.962	–	n.s.
FP16–INT8	–	−0.019	negligible
FP16–INT4	–	−0.002	negligible
INT8–INT4	–	−0.021	negligible

Table 8: ROUGE-L — Non-parametric tests across quantization levels.

Comparison	p-val.	δ	Interp.
Kruskal–Wallis (overall)	0.313	–	n.s.
FP16–INT8	–	0.083	negligible
FP16–INT4	–	0.078	negligible
INT8–INT4	–	0.005	negligible

6.1.4 *Effect Size Estimation.*

Effect size measures quantify the magnitude of differences between quantization levels, complementing statistical significance tests. Since both F1 and ROUGE-L metrics violated the assumption of normality (Section 6.1.2), non-parametric effect sizes were computed using Cliff's δ instead of Cohen's d.

Cliff's δ ranges from -1 to +1, where values close to 0 indicate negligible effects.

Table 9 reports the pairwise effect sizes between FP16, INT8, and INT4 for both metrics. All values are close to the 0, confirming that quantization has no practically relevant effect on task accuracy.

Table 9: Pairwise Cliff's δ effect sizes across quantization levels for F1 and ROUGE-L metrics.

Metric	Comparison	δ	Interpretation
F1	FP16–INT8	−0.019	negligible
F1	FP16–INT4	−0.002	negligible
F1	INT8–INT4	−0.021	negligible
ROUGE-L	FP16–INT8	0.083	negligible
ROUGE-L	FP16–INT4	0.078	negligible
ROUGE-L	INT8–INT4	0.005	negligible

These results confirm that differences between quantization levels, although measurable, are statistically and practically negligible. In other words, reducing precision from FP16 to INT8 or INT4 does not significantly alter model accuracy, supporting the robustness of quantization across both QA and summarization tasks.

6.2 Impact on Efficiency and Resource (RQ2-RQ5)

6.2.1 *Data Exploration (RQ2–RQ5).*

We first explored the efficiency and resource usage metrics across the three quantization levels (FP16, INT8, INT4) for Falcon, LLaMA, and Qwen models. The metrics considered include latency per token (s/token), peak memory usage (MB), CPU utilization (%), and energy consumption per token (J/token). Boxplots and density plots were used to visualize the distribution and variability of each metric.

The following tables 10, 11, 12 provide detailed summary statistics (mean, standard deviation, median, minimum, maximum, and variance) for each metric across quantization levels for Falcon, LLaMA, and Qwen models. These tables complement the visualizations by showing precise numerical values for each configuration. Figures 6, 8, and 10 show the boxplots of all efficiency and resource metrics across quantization levels for each model. Figures 7, 9, and 11 provide the corresponding density plots, illustrating distributional differences among FP16, INT8, and INT4.

Table 10: Summary statistics of Falcon metrics across quantization levels.

Metric	Quantization	Mean	SD	Median	Min	Max	Variance
Latency (s/token)	FP16	3.444	1.943	3.128	0.954	8.811	3.774
	INT8	1.991	0.973	1.698	0.634	4.856	0.948
	INT4	1.023	0.623	0.913	0.342	3.048	0.388
Memory (MB)	FP16	6489.509	0.241	6489.664	6489.082	6489.664	0.058
	INT8	3599.536	0.083	3599.582	3599.281	3599.582	0.007
	INT4	3417.000	0.017	3416.984	3416.984	3417.020	0.0003
CPU (%)	FP16	78.789	3.232	79.686	70.356	84.159	10.448
	INT8	78.701	3.860	79.511	68.768	84.686	14.899
	INT4	70.599	5.350	71.752	54.568	80.030	28.627
Energy (J/token)	FP16	0.807	0.603	0.682	0.026	2.222	0.363
	INT8	0.401	0.290	0.376	0.019	1.122	0.084
	INT4	0.801	1.225	0.285	0.010	4.576	1.502

Table 11: Summary statistics of LLaMA metrics across quantization levels.

Metric	Quantization	Mean	SD	Median	Min	Max	Variance
Latency (s/token)	FP16	2.706	2.212	1.981	0.982	11.765	4.893
	INT8	1.499	0.697	1.307	0.631	4.025	0.486
	INT4	1.022	0.722	0.805	0.438	3.744	0.521
Memory (MB)	FP16	6525.159	0.050	6525.168	6525.043	6525.195	0.003
	INT8	3649.590	0.060	3649.645	3649.488	3649.645	0.004
	INT4	3690.871	0.012	3690.879	3690.855	3690.887	0.0001
CPU (%)	FP16	77.987	2.925	79.031	70.461	82.278	8.555
	INT8	77.967	4.246	78.884	67.058	83.780	18.029
	INT4	70.675	4.350	71.917	59.269	78.557	18.920
Energy (J/token)	FP16	2.165	2.040	1.483	0.040	7.004	4.160
	INT8	0.321	0.191	0.336	0.021	0.852	0.037
	INT4	0.207	0.155	0.176	0.011	0.627	0.024

Table 12: Summary statistics of Qwen metrics across quantization levels.

Metric	Quantization	Mean	SD	Median	Min	Max	Variance
Latency (s/token)	FP16	2.342	1.477	1.914	0.921	7.923	2.183
	INT8	1.634	1.221	1.257	0.624	7.959	1.491
	INT4	0.943	0.369	0.887	0.406	1.955	0.136
Memory (MB)	FP16	6112.989	0.022	6113.004	6112.957	6113.004	0.0005
	INT8	3349.235	0.097	3349.152	3349.152	3349.348	0.0095
	INT4	3400.129	0.000	3400.129	3400.129	3400.129	0.0000
CPU (%)	FP16	80.261	3.258	80.913	71.154	85.827	10.618
	INT8	80.562	3.227	81.328	72.384	85.419	10.412
	INT4	71.097	5.780	73.207	57.115	78.671	33.403
Energy (J/token)	FP16	0.599	0.406	0.458	0.019	1.704	0.165
	INT8	0.367	0.301	0.262	0.013	1.207	0.091
	INT4	0.203	0.161	0.174	0.008	0.628	0.026

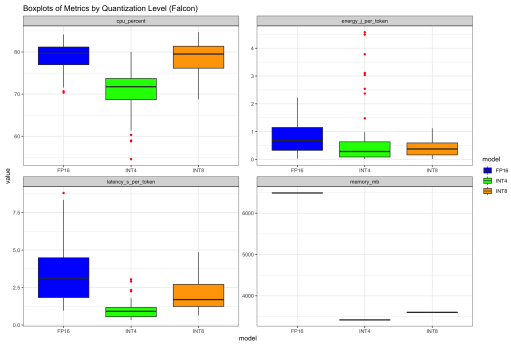

Figure 6: Boxplots of latency, memory, CPU, and energy per token for Falcon under FP16, INT8, and INT4 quantization.

Latency (RQ2).
Across all three models, latency decreases steadily as quantization becomes more aggressive, with INT4 showing the lowest latency and smallest variation. Density plots exhibit sharper peaks for INT4.

Memory Usage (RQ3).
Memory usage drops significantly from FP16 to INT8, with only marginal change between INT8 and INT4. Variability remains minimal across all models. Boxplots show narrow interquartile ranges,

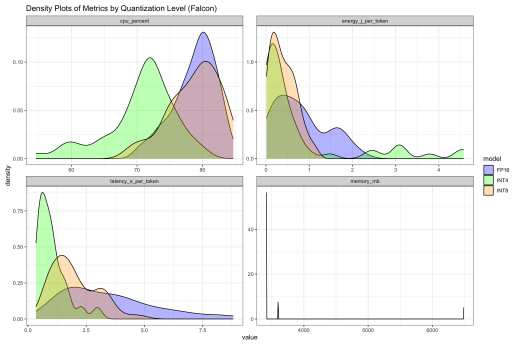

Figure 7: Density plots of latency, memory, CPU, and energy per token for Falcon under FP16, INT8, and INT4 quantization.

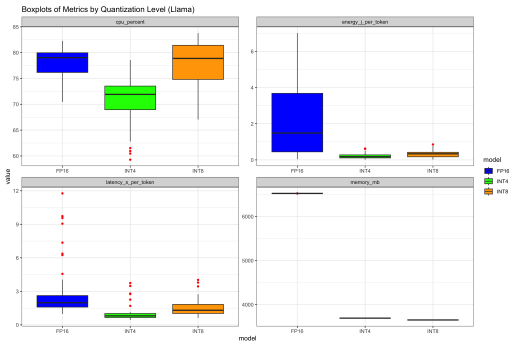

Figure 8: Boxplots of latency, memory, CPU, and energy per token for LLaMA under FP16, INT8, and INT4 quantization.

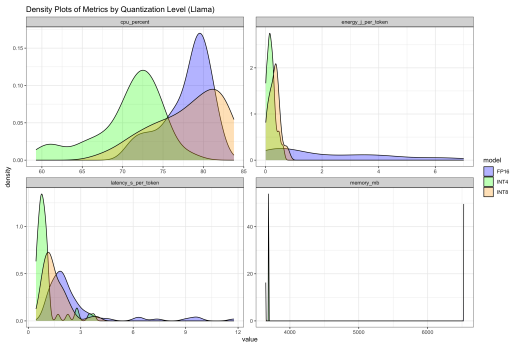

Figure 9: Density plots of latency, memory, CPU, and energy per token for LLaMA under FP16, INT8, and INT4 quantization.

and density curves are compact and unimodal across quantization levels.

CPU Utilization (RQ4).
CPU utilization is similar for FP16 and INT8 but decreases under INT4. Across models, distributions for INT4 show lower median and tighter spread compared to FP16 and INT8.

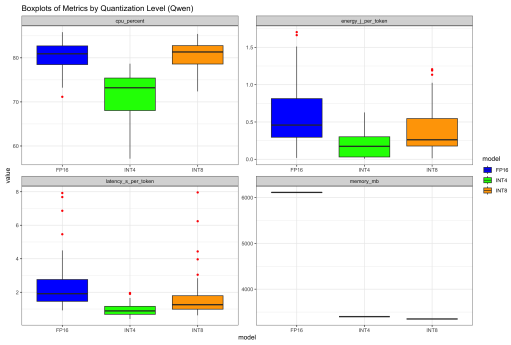

Figure 10: Boxplots of latency, memory, CPU, and energy per token for Qwen under FP16, INT8, and INT4 quantization.

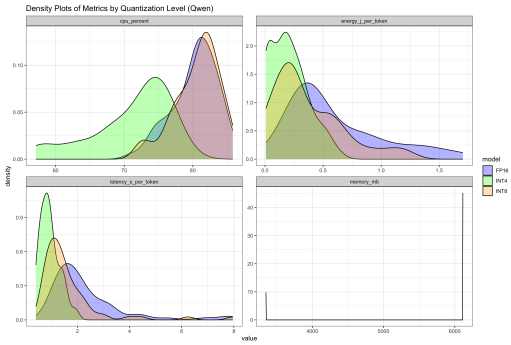

Figure 11: Density plots of latency, memory, CPU, and energy per token for Qwen under FP16, INT8, and INT4 quantization.

Energy Consumption (RQ5).
Energy consumption generally follows latency trends—INT8 and INT4 both reduce energy per token compared to FP16. Density plots show a clear leftward shift for INT8 and INT4.

6.2.2 *Normality Checks.*
We assessed the normality of each efficiency metric (latency, memory, CPU, energy) for each model using the Shapiro-Wilk test. Across all three models, most metrics did not follow a normal distribution even after transformations, with the exception of Falcon's latency per token, which was approximately normal. Therefore, we applied parametric and non-parametric tests according to the post-transformation normality of each metric.

6.2.3 *Hypothesis Testing.*
To examine differences among FP16, INT8, and INT4 quantization levels, we applied one-way ANOVA for metrics approximately normal (Falcon latency) and Kruskal-Wallis tests for non-normal metrics. Post-hoc pairwise comparisons were conducted using t-tests with Bonferroni correction for parametric cases, or Dunn's test with Bonferroni correction for non-parametric cases. Results are summarized below:

- **Falcon:**
  - **Latency:** Parametric t-tests revealed statistically significant differences across all quantization levels ($p < 0.001$), with large effect sizes (Cohen's $d = 0.95$–2.15).
  - **Memory:** Non-parametric tests indicated significant differences for all pairwise comparisons ($\delta = 1$).
  - **CPU utilization:** Significant differences occurred primarily between INT4 and other levels, with large effect sizes ($\delta = 0.80$–0.85).
  - **Energy consumption:** Non-parametric tests showed significant differences for FP16 vs INT4/INT8, with moderate effect sizes ($\delta = 0.30$–0.39).
- **LLaMA:**
  - **Latency:** Non-parametric Dunn tests showed significant differences across all quantization levels ($p < 0.001$), effect sizes $\delta = 0.52$–0.82.
  - **Memory:** All pairwise comparisons were significant ($p < 0.001$) with maximal effect sizes ($\delta = 1$).
  - **CPU utilization:** Significant differences mainly for INT4 vs FP16/INT8, large effect sizes ($\delta = 0.76$–0.86).
  - **Energy consumption:** Significant differences observed, effect sizes $\delta = 0.37$–0.82.
- **Qwen:**
  - **Latency:** Non-parametric Dunn tests revealed significant differences for all pairs ($p < 0.001$), effect sizes $\delta = 0.46$–0.85.
  - **Memory:** All comparisons significant, maximal effect sizes ($\delta = 1$).
  - **CPU utilization:** Significant differences for INT4 vs FP16/INT8, effect sizes $\delta = 0.89$–0.90.
  - **Energy consumption:** Significant differences for FP16 vs INT4/INT8, effect sizes $\delta = 0.35$–0.69.

Overall, these results consistently reject the null hypothesis H_0^2, indicating that quantization significantly affects all efficiency metrics across models. Parametric results report Cohen's d for effect size, while non-parametric results report Cliff's δ.

6.2.4 *Effect Size Estimation.*
Effect sizes for all significant pairwise comparisons are summarized in Table 13. Cohen's d is used for Falcon latency, while all other metrics use Cliff's δ.

Table 13: Pairwise comparisons of efficiency metrics across quantization levels. Effect sizes: Cohen's d (parametric) or Cliff's δ (non-parametric).

Metric	Model	Comparison	Adjusted p-value	Effect Size
Latency	Falcon	FP16 vs INT4	4.76e-25	2.15
Latency	Falcon	FP16 vs INT8	2.35e-07	0.95
Latency	Falcon	INT8 vs INT4	2.05e-10	1.36
Memory	Falcon	FP16 vs INT4	7.37e-38	1.00
Memory	Falcon	FP16 vs INT8	2.60e-14	1.00
Memory	Falcon	INT4 vs INT8	3.09e-08	-1.00
CPU	Falcon	FP16 vs INT4	2.98e-14	0.85
CPU	Falcon	FP16 vs INT8	1.00	-0.02
CPU	Falcon	INT4 vs INT8	8.92e-14	-0.80
Energy	Falcon	FP16 vs INT4	3.30e-03	0.30
Energy	Falcon	FP16 vs INT8	7.39e-04	0.39
Energy	Falcon	INT4 vs INT8	1.00	-0.04
Latency	LLaMA	FP16 vs INT4	1.98e-17	0.82
Latency	LLaMA	FP16 vs INT8	2.09e-05	0.52
Latency	LLaMA	INT4 vs INT8	2.14e-05	-0.58
Memory	LLaMA	FP16 vs INT4	7.30e-09	1.00
Memory	LLaMA	FP16 vs INT8	4.23e-38	1.00
Memory	LLaMA	INT4 vs INT8	4.77e-09	1.00
CPU	LLaMA	FP16 vs INT4	2.81e-13	0.86
CPU	LLaMA	FP16 vs INT8	1.00	-0.07
CPU	LLaMA	INT4 vs INT8	4.61e-14	-0.76
Energy	LLaMA	FP16 vs INT4	1.09e-16	0.82
Energy	LLaMA	FP16 vs INT8	1.26e-09	0.68
Energy	LLaMA	INT4 vs INT8	2.28e-02	-0.37
Latency	Qwen	FP16 vs INT4	8.44e-18	0.85
Latency	Qwen	FP16 vs INT8	1.28e-04	0.46
Latency	Qwen	INT4 vs INT8	6.90e-06	-0.53
Memory	Qwen	FP16 vs INT4	1.71e-10	1.00
Memory	Qwen	FP16 vs INT8	1.47e-40	1.00
Memory	Qwen	INT4 vs INT8	6.00e-11	1.00
CPU	Qwen	FP16 vs INT4	1.16e-16	0.89
CPU	Qwen	FP16 vs INT8	1.00	-0.06
CPU	Qwen	INT4 vs INT8	1.12e-18	-0.90
Energy	Qwen	FP16 vs INT4	1.88e-11	0.69
Energy	Qwen	FP16 vs INT8	5.82e-04	0.39
Energy	Qwen	INT4 vs INT8	3.73e-03	-0.35

7 DISCUSSION

This section interprets the results by linking observed metrics to quantization effects across models and tasks, integrating findings based on research questions and tarde-offs.

RQ1 investigated whether quantization compromises model accuracy. The results demonstrate that accuracy remains largely stable across all quantization levels, with only marginal decreases observed. In QA tasks, F1 scores declined by roughly 3%, while for Summarization tasks, ROUGE-L decreased by about 1%. These variations are small enough to be statistically insignificant, confirming that post-training quantization preserves task performance even under aggressive compression. Summarization tasks appear particularly robust to precision reduction, while QA performance remains strong, suggesting that lower-bit models such as INT8 or INT4 can be used in real-world mobile applications without sacrificing accuracy.

RQ2 focused on inference latency. The progressive reduction in inference latency from FP16 through INT8 to INT4 quantization demonstrates a clear relationship between precision reduction and computational efficiency gains. The substantial improvements observed at INT8 validate this quantization level as a practical optimization for latency-sensitive mobile applications. The further transition to INT4, while continuing the downward trend, reveals diminishing marginal returns indicating that computational bottlenecks increasingly shift from arithmetic operations to memory bandwidth constraints at extreme precision levels. The consistently lower variance observed in INT4 configurations across all evaluated models suggests more predictable inference behavior at aggressive quantization levels. For mobile deployment scenarios where consistent user experience is crucial, this predictability represents a valuable characteristic beyond raw performance metrics. However, the modest incremental gains between INT8 and INT4 suggest that 8-bit precision may represent an optimal balance point for most latency-critical applications.

RQ3 addressed memory usage, revealing one of the most dramatic improvements from quantization. Peak RAM consumption dropped by approximately 75% when moving from FP16 to INT8 and by around 80% at INT4. The minimal variance across all configurations underscores the predictability of memory requirements under quantization. The small gap between INT8 and INT4 memory savings—despite halving bit width—reveals that runtime buffers, activations, and framework overheads dominate memory usage beyond a certain point. Thus, if memory constraints allow INT8 deployment, the incremental gain from INT4 may not justify its added complexity. The stability of these reductions also simplifies resource allocation and model selection for developers targeting memory-limited mobile environments.

CPU utilization, examined RQ4 shows a more nuanced relationship with quantization. FP16 and INT8 exhibit similar processor loads, indicating that reduced precision does not automatically lead to proportionally lower computational demand. This outcome likely reflects dequantization overhead and limited hardware support for optimized integer arithmetic on mobile CPUs. However, INT4 consistently reduces CPU activity across models, suggesting that at sufficiently aggressive quantization, computation becomes lighter and better aligned with processor efficiency. This trend implies that extreme quantization may enable more efficient vectorization or improved cache usage, though at the potential cost of greater variability in processing characteristics. Overall, quantization alleviates CPU load, but the extent of this benefit depends strongly on processor architecture and software optimization for low-bit arithmetic.

RQ5 explored energy consumption, which closely followed latency trends, confirming the direct relationship between inference duration and power draw on mobile devices. Both INT8 and INT4 quantization deliver substantial efficiency gains, with density distributions exhibiting pronounced leftward shifts that indicate not only reduced mean consumption but also more concentrated, predictable energy profiles. The moderate effect sizes observed in INT8-to-INT4 transitions align with diminishing returns identified in latency analysis, reinforcing the interpretation that memory bandwidth rather than computational intensity becomes the limiting factor at extreme quantization levels. This pattern suggests that beyond 8-bit precision, the primary constraint on energy efficiency shifts from computational workload to data movement overhead therefore INT8 provides an optimal balance in that regard.

Additionally, the observed results reveal several meaningful trade-offs highlighting how quantization reshapes the balance between performance and sustainability. Firstly, the relationship between accuracy and efficiency reveals a highly favorable outcome. Quantization maintains nearly identical accuracy while significantly improving latency and energy metrics. This asymmetric improvement, large efficiency gains with minimal accuracy cost,

validates quantization as a key optimization strategy. Secondly, the comparison of resource utilization and performance demonstrates that quantization delivers complementary benefits across multiple system dimensions. INT8 achieves major reductions in memory usage and latency, making previously infeasible deployments possible. While CPU utilization behaves differently, INT4 configurations show lower average usage, suggesting that aggressive quantization can further alleviate computational load. Therefore, quantization improves system efficiency holistically, optimizing both computational and memory resources. When considering all metrics together from a sustainability and feasibility perspective considering trade-offs, particularly INT8 emerges as a sustainable and feasible approach for mobile deployment. Our results indicate that efficiency metrics move together and this convergence suggests that quantization reshapes the efficiency–accuracy landscape toward balanced sustainability.

Lastly, from a practical standpoint, the results offer several actionable insights. For developers, INT8 quantization should be the default configuration when deploying LLMs on mobile or resource-limited platforms as it provides substantial improvements without compromising quality. For researchers, the diminishing returns between INT8 and INT4 suggest that future work should focus on optimizing inference runtimes and mixed-precision techniques rather than pursuing more aggressive quantization. For device manufacturers, the results highlight the importance of enhancing mobile CPUs to better utilize low-bit arithmetic. Finally, from a sustainability perspective, quantization directly contributes to greener AI by lowering energy consumption per task, making on-device inference both efficient and environmentally responsible.

8 THREATS TO VALIDITY

8.1 Internal Validity

Our study faces several internal validity threats. First, human-in-the-loop configuration of inference flags can introduce systematic differences across runs; to reduce this, we fixed commonly used settings (e.g., -c 2096, –n-predict 2048, –temp 0.8, –single-turn, –no-display-prompt) and kept them constant for reproducibility. Second, task selection may bias estimates toward models with smaller context windows; we mitigated this by extracting homogeneous subsets matched on token counts from the original dataset. Third, instrumentation limits affect causal claims: energy and CPU metrics may vary across platforms, and energy derived from battery telemetry is less precise than power-meter measurements. Fourth, environment variability can confound treatment effects; we enforced a stable setup (phone on charge, background services off, notifications and Bluetooth disabled, same network). Finally, we controlled maturation and order effects via warmup, cooldown intervals, and randomization of tasks within device-specific blocks.

8.2 External Validity

External validity is limited by the specific hardware setup and subject selection used in this study. The experiments run on a Google Pixel 9 as the execution device and a Raspberry Pi 4 as the orchestration host, which may not represent other smartphones, SoCs, thermal envelopes, or power-management behaviors that could influence latency and energy profiles across devices. The reliance on this single phone model and a specific orchestration board may constrain generalizability to different Android versions, iOS devices, or commodity edge hardware. Model selection was bounded by domain goals and time constraints, which can narrow coverage of architectures, training data mixtures, context lengths, and quantization schemes, potentially biasing external conclusions. To partially mitigate these risks, subjects were drawn from widely used open-source 3B-class models available in ready-to-run formats, aligning with typical on-device developer usage. Future work should expand device diversity, include alternative deployment platforms, and sample broader model families and parameter scales.

8.3 Construct Validity

We grouped tasks into three simple buckets: Question and Answer, Short Summary, and Long Summary, to represent different content types in a transparent way. Length thresholds for short vs long were chosen using dataset statistics from our script, so the split is data-driven rather than hand-picked. Still, these buckets may miss edge cases or styles that do not fit cleanly, which can limit how well our measures capture the construct. We also used a constant token-to-words conversion; this is practical but can introduce error when tokenizers behave differently or when language varies. To strengthen construct validity next, we can test finer length bands, calibrate word conversion per tokenizer, and validate boundaries with small human checks.

8.4 Conclusion Validity

We relied on non-parametric inference for virtually all outcomes and configurations, acknowledging its lower statistical power compared to parametric tests. Accuracy (F1/ROUGE-L), energy (J/token), CPU (%), memory (MB), and most latency distributions violated normality and/or homoscedasticity diagnostics; therefore, per metric we used Kruskal–Wallis across treatments (FP16, INT8, INT4) followed by Dunn's post-hoc pairwise comparisons with Holm correction for multiple testing. The single exception is latency for Falcon-3 (3B), which passed Shapiro–Wilk and variance checks; for that case we applied one-way ANOVA (Welch when variances were unequal) with Tukey HSD (or Games–Howell with Welch), reporting means, SD, and η^2 (or ω^2). This approach prioritizes validity under assumption violations while transparently trading off statistical power; where Gaussian structure holds (Falcon latency), we leverage parametric efficiency. Test families are predefined per metric to control Type I error.

9 CONCLUSIONS

The empirical analysis demonstrates that quantization enables substantial computational and energy savings while maintaining reliable language understanding and generation quality on mobile hardware. Across models and tasks, inference became consistently faster and lighter, with no statistically significant loss in accuracy. These outcomes confirm that precision reduction can serve not only as an efficiency measure but also as a viable path toward sustainable and privacy-respecting on-device LLM deployment. Rather than diminishing model capability, quantization redefines the operational boundary of LLMs by aligning performance demands with realistic mobile constraints.

Future research could extend this work by exploring adaptive or mixed-precision quantization strategies that dynamically adjust bit widths, integrating quantization with complementary compression methods such as pruning or distillation, and evaluating performance across heterogeneous mobile processors and hardware accelerators. Broader task coverage, longer context windows, and real-world user scenarios would further strengthen the generalizability of these findings and guide the design of next-generation sustainable LLM systems.

10 TIME LOG LINK

The time log table can be found **here**

11 RUN TABLE

The run table can be found **here**

12 OVERLEAF TEMPLATE

Link to overleaf template can be found **here**

REFERENCES

[1] Z. Wang, Z. Chu, T. V. Doan, S. Ni, M. Yang, and W. Zhang, "History, development, and principles of large language models: an introductory survey," *AI and Ethics*, vol. 5, no. 3, pp. 1955–1971, 2025.

[2] N. Anton, "Online vs. offline llm inference: Unlocking the best of both worlds in mobile applications," *Universum:* , vol. 10, no. 11 (128), pp. 43–46, 2024.

[3] A. Novikau, "Online vs. offline llm inference: Unlocking the best of both worlds in mobile applications," *IJSEA| International Journal of Science and Engineering Applications*, vol. 1, p. 5, 2024.

[4] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," *arXiv preprint arXiv:1909.08053*, 2019.

[5] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th symposium on operating systems principles*, 2023, pp. 611–626.

[6] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "Awq: Activation-aware weight quantization for on-device llm compression and acceleration," *Proceedings of machine learning and systems*, vol. 6, pp. 87–100, 2024.

[7] Hugging Face Community, "Llama 7b gpu memory requirement," https://discuss.huggingface.co/t/llama-7b-gpu-memory-requirement/34323, 2023, accessed: 2025-09-10.

[8] Enclave AI, "Understanding llm model sizes," https://enclaveai.app/blog/2024/05/13/understanding-llm-model-sizes/, 2024, accessed: 2025-09-10.

[9] S. Baltes, F. Angermeir, C. Arora, M. M. Barón, C. Chen, L. Böhme, F. Calefato, N. Ernst, D. Falessi, B. Fitzgerald *et al.*, "Evaluation guidelines for empirical studies in software engineering involving llms," *arXiv preprint arXiv:2508.15503*, 2025.

[10] J. Lang, Z. Guo, and S. Huang, "A comprehensive study on quantization techniques for large language models," in *2024 4th International Conference on Artificial Intelligence, Robotics, and Communication (ICAIRC)*, 2024, pp. 224–231.

[11] L. Wei, Z. Ma, C. Yang, and Q. Yao, "Advances in the neural network quantization: A comprehensive review," *Applied Sciences*, vol. 14, no. 17, 2024. [Online]. Available: https://www.mdpi.com/2076-3417/14/17/7445

[12] Meta AI, "Llama 3.2: Revolutionizing edge AI and vision with open, customizable models," https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/, 2024, accessed 2025-10-22.

[13] Q. Team, "Qwen2.5: A party of foundation models," September 2024. [Online]. Available: https://qwenlm.github.io/blog/qwen2.5/

[14] Falcon-LLM Team, "The falcon 3 family of open models," https://huggingface.co/blog/falcon3, December 2024, falcon3 family overview; includes Falcon3-3B model details.

[15] M. Monteiro Santos, A. Barros, L. Rodrigues, D. Dermeval, T. Primo, I. Ibert, and S. Isotani, "Near feasibility, distant practicality: Empirical analysis of deploying and using llms on resource-constrained smartphones," ser. ICTD '24. New York, NY, USA: Association for Computing Machinery, 2025. [Online]. Available: https://doi.org/10.1145/3700794.3700817

[16] X. Yan and Y. Ding, "Are we there yet? a measurement study of efficiency for llm applications on mobile devices," ser. FMSys. New York, NY, USA: Association for Computing Machinery, 2025. [Online]. Available: https://doi.org/10.1145/3722565.3727192

[17] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "Awq: Activation-aware weight quantization for llm compression and acceleration," 2024. [Online]. Available: https://arxiv.org/abs/2306.00978

[18] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "Gptq: Accurate post-training quantization for generative pre-trained transformers," *arXiv preprint arXiv:2210.17323*, 2023. [Online]. Available: https://arxiv.org/abs/2210.17323

[19] T. Dettmers, M. Lewis, S. Shleifer, and L. Zettlemoyer, "Llm.int8(): 8-bit matrix multiplication for transformers at scale," *arXiv preprint arXiv:2208.07339*, 2022. [Online]. Available: https://arxiv.org/abs/2208.07339

[20] G. Xiao, J. Lin, Y. Lou, and S. Han, "Smoothquant: Accurate and efficient post-training quantization for large language models," *arXiv preprint arXiv:2211.10438*, 2022. [Online]. Available: https://arxiv.org/abs/2211.10438

[21] L. Fan, L. Li, Z. Ma, S. Lee, H. Yu, and L. Hemphill, "A bibliometric review of large language models research from 2017 to 2023," *ACM Transactions on Intelligent Systems and Technology*, vol. 15, no. 5, pp. 1–25, 2024.

[22] LMArena, "Text arena from lmarena.ai," https://lmarena.ai/leaderboard/text, 2025, accessed: 2025-09-29.

[23] H. Touvron, L. Martin, K. Stone *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023. [Online]. Available: https://arxiv.org/abs/2307.09288

[24] A. Jiang and M. A. Team, "Mistral 7b," *arXiv preprint arXiv:2310.06825*, 2023, preprint. [Online]. Available: https://arxiv.org/abs/2310.06825

[25] E. Almazrouei, H. Alobeidli, A. Alshamsi *et al.*, "Falcon-7b/40b: Building an open large language model," *Technical report, Technology Innovation Institute (TII)*, 2023, see also arXiv:2311.16867 for FalconRef and official model card. [Online]. Available: https://huggingface.co/tiiuae

[26] Hugging Face Community, "Dataset news qa summarization from huggingface," https://huggingface.co/datasets/glnmario/news-qa-summarization/viewer/default/train?views%5B%5D=train&row=28, 2025, accessed: 2025-09-29.

[27] G. Raffin and D. Trystram, "Dissecting the software-based measurement of cpu energy consumption: a comparative analysis," *IEEE Transactions on Parallel and Distributed Systems*, 2024.

[28] NIST/SEMATECH, "Engineering statistics handbook: 1.3.3.7. boxplots," https://www.itl.nist.gov/div898/handbook/prc/section1/prc16.htm, 2012, defines boxplot construction and Tukey fences at Q1±1.5IQR Q1±1.5IQR; widely cited technical reference.

[29] J. W. Tukey, *Exploratory Data Analysis*. Reading, MA: Addison-Wesley, 1977, introduces boxplots and the 1.5,×,IQR rule for identifying outliers.

[30] E. González-Estrada and W. Cosmes, "Shapiro–wilk test for skew normal distributions based on data transformations," *Journal of Statistical Computation and Simulation*, vol. 89, no. 17, pp. 3258–3272, 2019.

[31] S. Vegas, "Analyzing software engineering experiments: everything you always wanted to know but were afraid to ask," in *Proceedings of the 40th International Conference on Software Engineering: Companion Proceeedings*, ser. ICSE '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 534–535. [Online]. Available: https://doi.org/10.1145/3183440.3183466